

C Dataset details

Among the nine datasets in our experiment, three were not originally designed for in-context learning in natural language processing. Here, we provide further details on the data processing:

- **COGS:** The original COGS dataset (Kim & Linzen, 2020) evaluates the compositional generalization of machine learning models through a task that introduces compositional distribution shifts in input-output mappings. In this study, we use the test dataset, sampling 10 entries as in-context demonstrations.
- **List Function:** The original work designed these functions to investigate the human-like learning abilities of cognitive systems (Rule, 2020). Subsequent studies have explored LLMs’ capabilities in rule induction (pattern inference) (Qiu et al., 2024; Li et al., 2025) and in-context learning (output prediction) (Zheng et al., 2025a). In this work, we adopt the processed dataset from Li et al. (2025).
- **RAVEN:** The original RAVEN dataset (Zhang et al., 2019) assesses the analogical reasoning abilities of visual models using images of symbols. We adopt the abstracted lm-RAVEN dataset (Hu et al., 2023), which tokenizes image attributes into symbolic matrices.

D Prompt templates

In this section, we include our prompt templates as follows: prompt for dataset-specific instructions, prompt for reasoning frameworks, and prompt used in our tailored experiment.

D.1 Prompt for dataset-specific instructions

Prompt Templates

ARC-AGI / MiniARC / 1D-ARC / COGS

Below are several examples of input and output grids/lists.
There exists an underlying pattern/function that maps the input grid/list to the output grid/list.

<in-context demonstrations>

Your task is to predict the output grid/list based on the new input grid/list:

<test input>

SCAN

Below are several examples that convert natural language commands into action sequences.

<in-context demonstrations>

Your task is to predict the output sequence based on the new input sequence:

<test input>

MiniSCAN

Here is a task: Your task is to convert an input sequence into an output sequence based on specific rules.
Each word in the input sequence either corresponds to a direct transformation into a word in the output sequence, or a rule that defines how the output sequence should be structured.

<in-context demonstrations>

Your task is to predict the output sequence based on the new input sequence:

<test input>

SALT

Below are several examples that convert english sentence into an output sequence based on specific rules.
Each word in the input sequence either corresponds to a translated word in the output, or indicates a syntactic rule (e.g., repeating or reordering semantic units) for forming the output sequence.

<in-context demonstrations>

Your task is to predict the output sequence based on the new english sentence:

<test input>

List Function

Below are several examples of input and output lists.
There exists an underlying python function that maps the input list to the output list.

<in-context demonstrations>

Your task is to predict the output list based on the new input list:

<test input>

RAVEN

Below are several rows of abstracted symbols. The symbols follow a certain rule or pattern.

<in-context demonstrations>

Your task is to predict the missing symbol based on the incomplete row:

<test input>

D.2 Prompt for reasoning frameworks

For a fair comparison of reasoning frameworks against vanilla zero-shot CoT and direct answering, we adopt a one-off prompting approach rather than a complex agent framework. For Tree-of-Thought, we use the prompt proposed by [Hulbert \(2023\)](#). For ReAct, we employ the prompt from Qwen’s implementation ([Bai et al., 2023](#)).

Prompt Templates

Direct Answering

<regular question instructions and data>

Please output your final answer in the following json dict format without any explanation:

```
{
  "answer": "your answer"
}
```

Chain-of-Thought

<regular question instructions and data>

Please first perform reasoning and then output your final answer in the following json dict format:

```
{
  "reasoning": "your reasoning process",
  "answer": "your answer"
}
```

ReAct

You should now solve the below question using the following pipeline:

Question: the input question you must answer
 Thought: Think about what to do
 Action: Your action process
 Observation: the result of the action
 (this Thought/Action/Observation can be repeated zero or more times)
 Thought: I now know the final answer
 Final Answer: the final answer to the original input question

<regular question instructions and data>

You should respond in the following json dict format:

```
{
  "process": "your full problem-solving process",
  "answer": "your final answer"
}
```

Tree-of-Thought

Imagine three different experts are answering this question.
 All experts will write down 1 step of their thinking,
 then share it with the group.
 Then all experts will go on to the next step, etc.
 If any expert realises they’re wrong at any point then they leave.

<regular question instructions and data>

You should respond in the following json dict format:

```
{
  "discussion": "full discussion and reasoning process of experts",
  "answer": "final agreed answer"
}
```